Práctica 2

1)INTERCAMBIO DIRECTO o BURBUJA

Tras medir los tiempos, rellenar la tabla:

Esto medido con un ordenador de procesador Intel i5-4590 3.30GHz:

n	t ordenado	t inverso	t aleatorio
10000	612	1463	1132
20000	2365	5765	4513
40000	9472	23059	18295
80000	37885	FdT	FdT
160000	FdT	FdT	FdT

(Las gráficas de los algoritmos empleados se encuentran en el apartado de "Sección de gráficas de algoritmos empleados", al final del documento).

Razone si los diferentes tiempos obtenidos concuerdan con lo esperado, según la complejidad temporal estudiada.

Estos van acordes con lo que sabemos de este método, pues, al método de ordenación por inserción directa ir recorriendo el array cada vez que encuentra un elemento menor hasta colocarlo en su sitio, es coherente que, dado un array inversamente ordenado, sea en este en el que más tiempo tarde al realizar esta operación la mayor cantidad de veces. Siendo el caso con un array ordenado el caso opuesto, al tenerlo mucho más fácil, y el de array aleatorio siendo más o menos y caso intermedio entre ambos.

2)SELECCIÓN

Implementar una clase SeleccionTiempos.java que tras ser ejecutada sirva para rellenar la siguiente tabla:

Esto medido con un ordenador de procesador Intel i5-4590 3.30GHz:

n	t ordenado	t inverso	t aleatorio
10000	518	590	518
20000	2028	2311	2046
40000	8247	9153	8230
80000	32662	36573	32501
160000	FdT	FdT	FdT

(Las gráficas de los algoritmos empleados se encuentran en el apartado de "Sección de gráficas de algoritmos empleados", al final del documento).

Razone si los diferentes tiempos obtenidos concuerdan con lo esperado, según la complejidad temporal estudiada.

Como ya sabemos, la complejidad de este algoritmo es, independientemente del caso, O(n^2), por lo que es coherente que los tiempos en los tres casos sean relativamente similares, variando ligeramente el del vector inverso.

3)INSERCIÓN

Implementar una clase InsercionTiempos.java que tras ser ejecutada sirva para rellenar la siguiente tabla:

Esto medido con un ordenador de procesador Intel i5-4590 3.30GHz:

n	t ordenado	t inverso	t aleatorio
10000	FdF	799	406
20000	FdF	3126	1575
40000	FdF	12466	6271
80000	FdF	49953	24887
160000	FdF	FdT	FdT
320000	FdF	FdT	FdT
640000	FdF	FdT	FdT
1280000	FdF	FdT	FdT
2560000	70	FdT	FdT
5120000	140	FdT	FdT
10240000	275	FdT	FdT
20480000	548	FdT	FdT
40960000	1081	FdT	FdT
81920000	2165	FdT	FdT

(Las gráficas de los algoritmos empleados se encuentran en el apartado de “Sección de gráficas de algoritmos empleados”, al final del documento).

Razone si los diferentes tiempos obtenidos concuerdan con lo esperado, según la complejidad temporal estudiada.

Aquí podemos ver una diferencia abismal entre el caso de ordenado y sus dos alternativas, esto es así pues la complejidad de este algoritmo puede ser o O(n) en el mejor de los casos, como se puede intuir siendo este el de t ordenado, o O(n^2) en los demás casos, de ahí que inverso y aleatorio sean relativamente similares.

4)RÁPIDO

Implementar una clase RapidoTiempos.java que tras ser ejecutada sirva para rellenar la siguiente tabla:

Esto medido con un ordenador de procesador Intel i5-4590 3.30GHz:

n	t ordenado	t inverso	t aleatorio
250000	117	138	177
500000	240	301	403
1000000	493	563	853
2000000	1036	1156	2078
4000000	2141	2396	5579
8000000	4450	4971	16937
16000000	9409	10332	56710

(Las gráficas de los algoritmos empleados se encuentran en el apartado de "Sección de gráficas de algoritmos empleados", al final del documento).

Razone si los diferentes tiempos obtenidos concuerdan con lo esperado, según la complejidad temporal estudiada.

Como podemos ver en los dos primeros, y a comienzos del aleatorio, los tiempos tiendes a ir multiplicándose por 2 y poco, lo cual cumple con la complejidad en el mejor de los casos de este algoritmo, la cual es O(n*logn).

Pero, sin embargo, es el caso aleatorio donde vemos que esta norma se rompe, pero, y esto es lo importante, solo en algunas iteraciones, lo cual se debe a que, por el comportamiento de este algoritmo a la hora de ir almacenando elementos de forma contigua, puede llegar a consumir más tiempo de lo esperado según sea más grande el vector que va creando.

5)RÁPIDO + INSERCIÓN

Reimplementar el método de inserción, para que en lugar de ordenar el vector que se le pasa en todas sus posiciones [0 .. longitud-1], lo ordene solo entre las posiciones [inferior .. superior], posiciones que se pasarán como parámetros.

Implementar una clase RapidoInsercion.java que ordene los elementos del vector según lo apuntado antes.

Ahora se propone hacer una toma de tiempos, que, aunque no sea concluyente y determinante para compararlos, sí será orientativa. Para ello, implementar una clase

RapidoInsercionTiempos.java que tomará tiempos (para un tamaño de 6 n=16 millones de elementos generados inicialmente de forma aleatoria) para rellenar la siguiente tabla:

Esto medido con un ordenador de procesador Intel Core i5 12400 2.5GHz y 16GB de RAM:

Caso	t aleatorio
Rápido	41573
Rápido + Inserción (k=5)	41983
Rápido + Inserción (k=10)	41670
Rápido + Inserción (k=20)	41126
Rápido + Inserción (k=30)	40600
Rápido + Inserción (k=50)	38012
Rápido + Inserción (k=100)	27680
Rápido + Inserción (k=200)	27508
Rápido + Inserción (k=500)	12587
Rápido + Inserción (k=1000)	21359

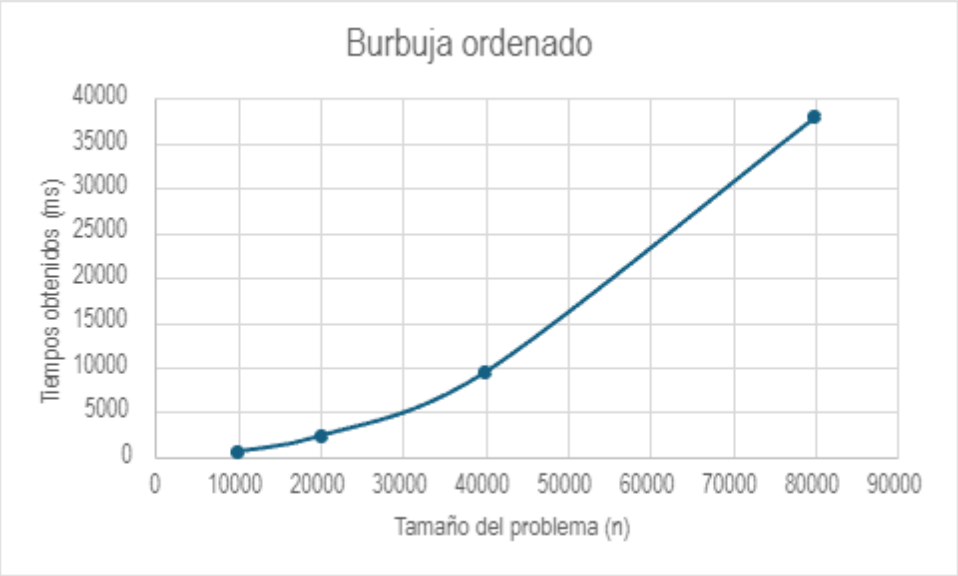
(Las gráficas de los algoritmos empleados se encuentran en el apartado de “Sección de gráficas de algoritmos empleados”, al final del documento).

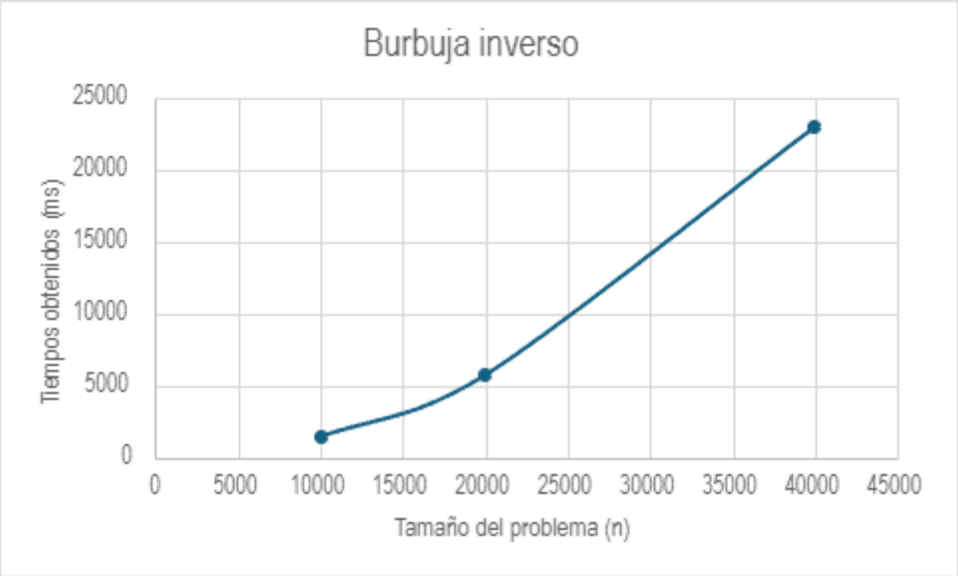
Razone conclusiones obtenidas a partir de la tabla anterior.

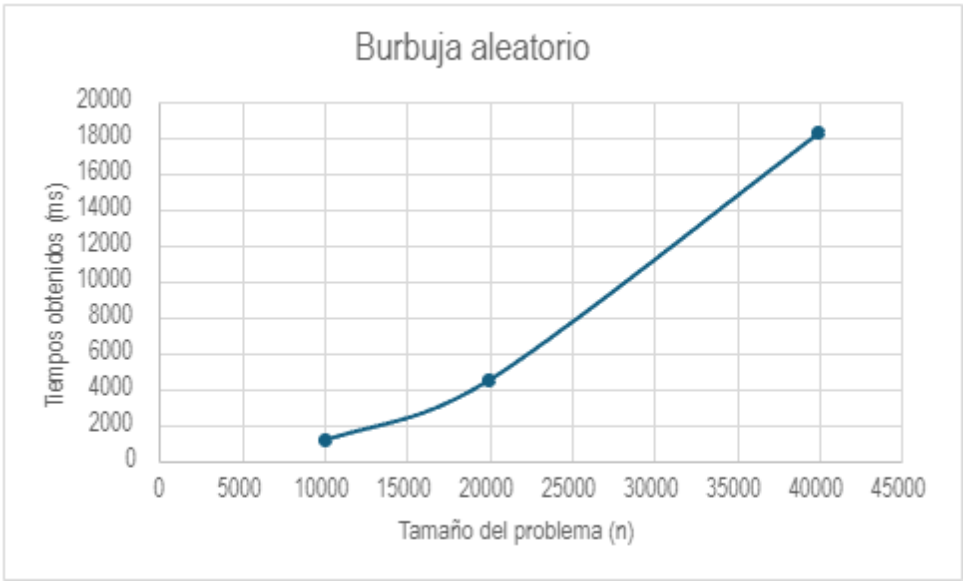
Según los resultados, podemos ver cómo, inicialmente, este algoritmo resulta mejor que el rápido común, obteniendo tiempos menores gracias a ir aplicando inserción a los subvectores creados, siendo la diferencia mayor según la k (el tamaño de los subvectores) va aumentando.

Sin embargo, podemos apreciar en la última toma de tiempo, para que, llegado a un tamaño de k, los resultados empiezan a empeorar, esto es así porque, al ser tan grande este valor, y por ende crearse subvectores de gran tamaño, la posibilidad de que estos estén ordenados se va haciendo menor, por lo que habría que ordenarlos en sí mismos, perdiendo bastante tiempo en ello.

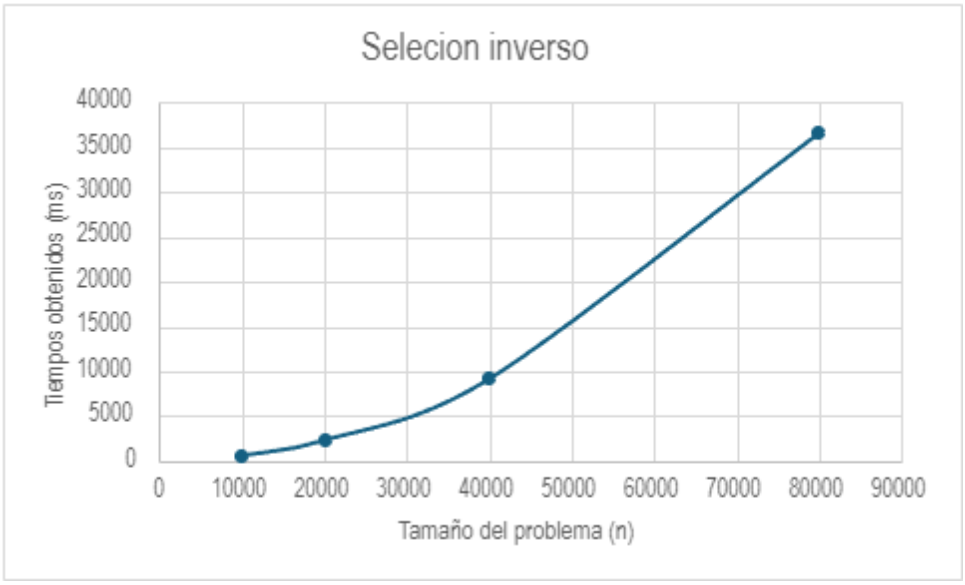
Sección de gráficas de los algoritmos empleados

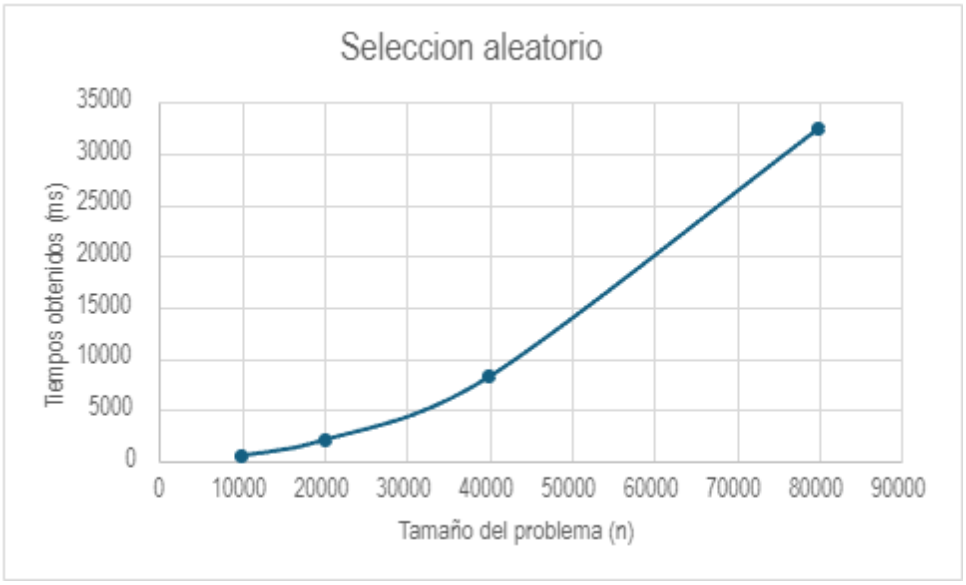




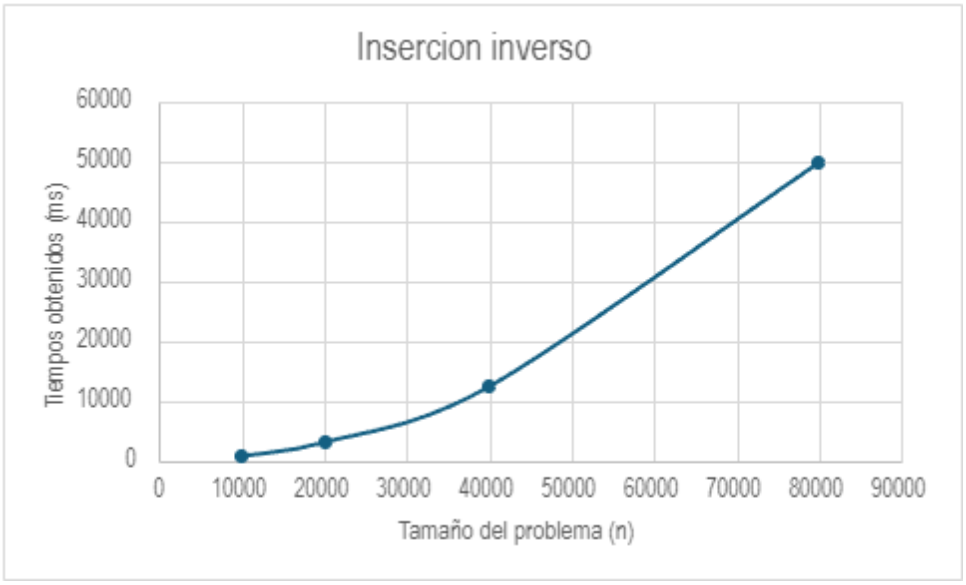

Burbuja aleatorio
Tiempos obtenidos (ms)
20000
18000
16000
14000
12000
10000
8000
6000
4000
2000
0
0
5000
10000
15000
20000
25000
30000
35000
40000
45000
Tamaño del problema (n)

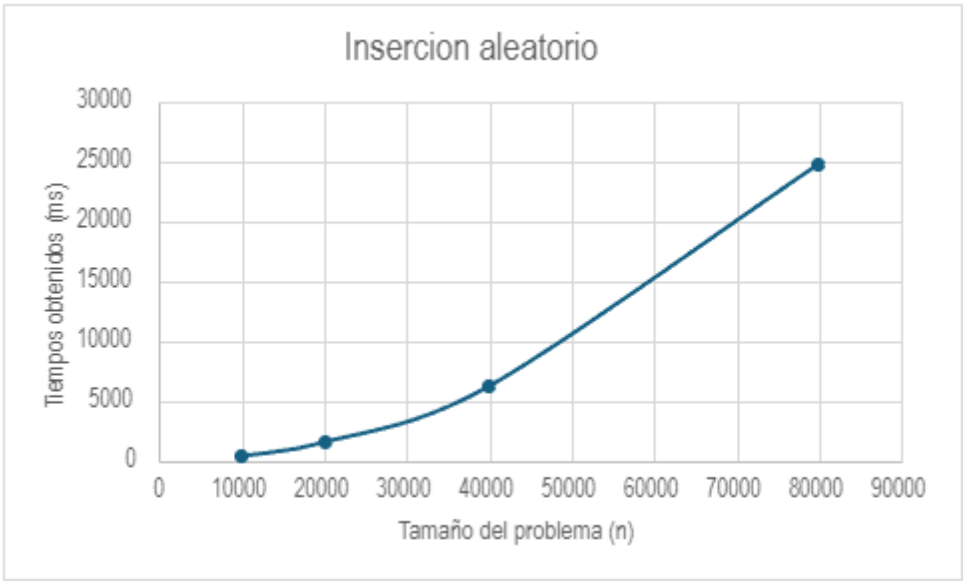

Seleccion ordenado
Tiempos obtenidos (ms)
35000
30000
25000
20000
15000
10000
5000
0
0
10000
20000
30000
40000
50000
60000
70000
80000
90000
Tamaño del problema (n)


Selecion inverso
Tiempos obtenidos (ms)
40000
35000
30000
25000
20000
15000
10000
5000
0
0
10000
20000
30000
40000
50000
60000
70000
80000
90000
Tamaño del problema (n)

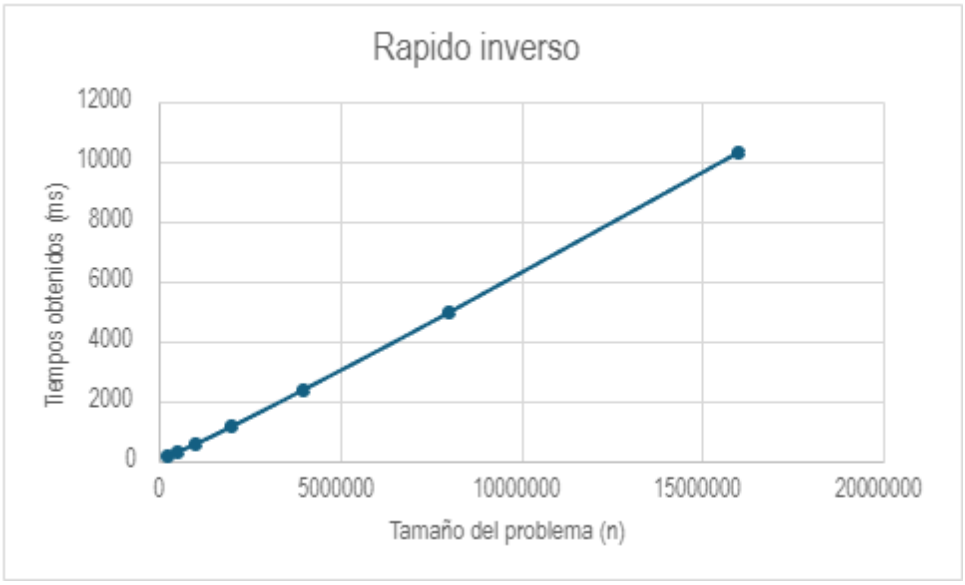

Seleccion aleatorio
Tiempos obtenidos (ms)
35000
30000
25000
20000
15000
10000
5000
0
0
10000
20000
30000
40000
50000
60000
70000
80000
90000
Tamaño del problema (n)

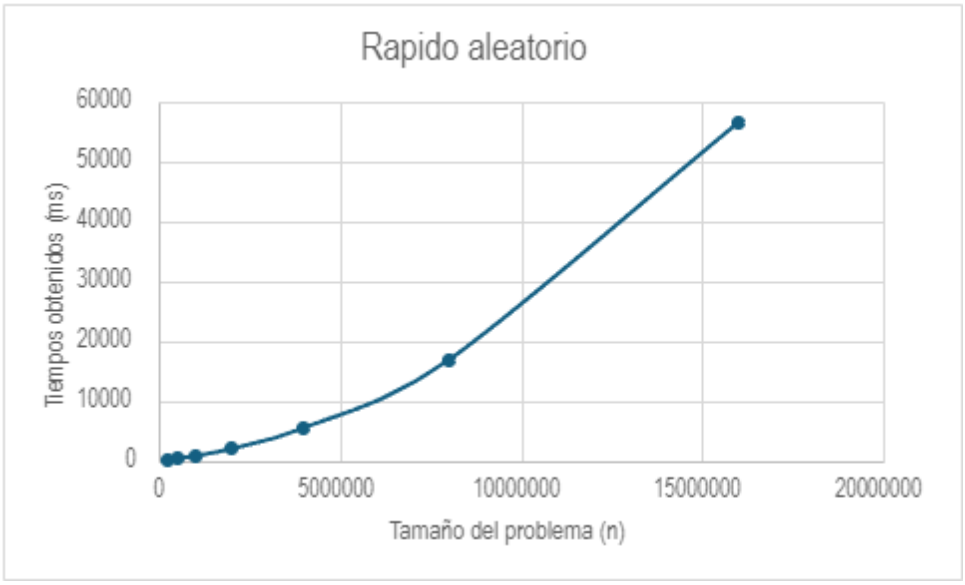

Inserccion ordenado
Tiempos obtenidos (ms)
2500
2000
1500
1000
500
0
0
20000000
40000000
60000000
80000000
100000000
Tamaño del problema (n)

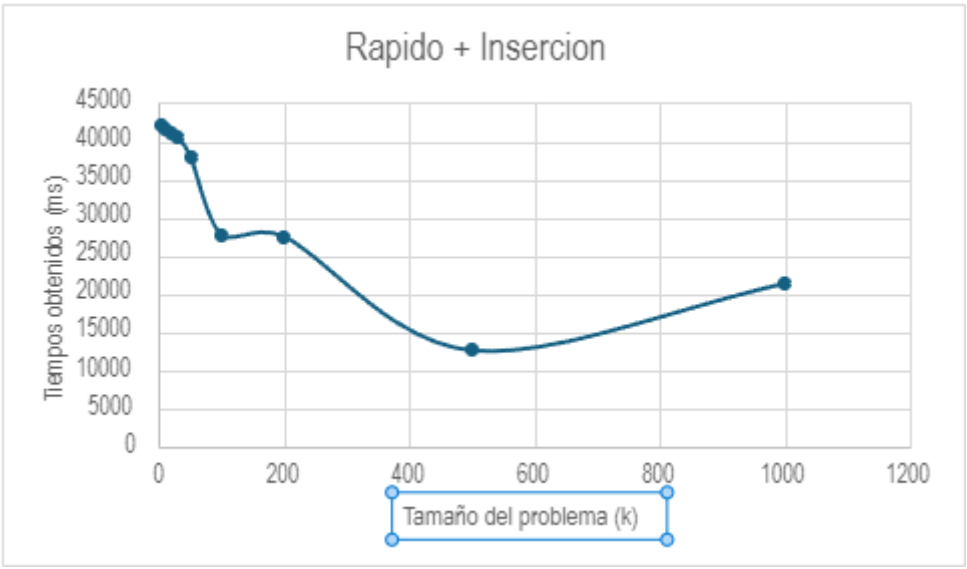

Insercion inverso
Tiempos obtenidos (ms)
60000
50000
40000
30000
20000
10000
0
0
10000
20000
30000
40000
50000
60000
70000
80000
90000
Tamaño del problema (n)


Insercion aleatorio
Tiempos obtenidos (ms)
30000
25000
20000
15000
10000
5000
0
0
10000
20000
30000
40000
50000
60000
70000
80000
90000
Tamaño del problema (n)


Rapido ordenado
Tiempos obtenidos (ms)
10000
9000
8000
7000
6000
5000
4000
3000
2000
1000
0
0
5000000
10000000
15000000
20000000
Tamaño del problema (n)


Rapido inverso
Tiempos obtenidos (ms)
12000
10000
8000
6000
4000
2000
0
0
5000000
10000000
15000000
20000000
Tamaño del problema (n)


Rapido aleatorio
60000
50000
40000
30000
20000
10000
0
Tiempos obtenidos (ms)
0
5000000
10000000
15000000
20000000
Tamaño del problema (n)


Rapido + Insercion
45000
40000
35000
30000
25000
20000
15000
10000
5000
0
Tiempos obtenidos (ms)
0
200
400
600
800
1000
1200
Tamaño del problema (k)